HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

ĐỀ TÀI: XÂY DỰNG WEBSITE ĐỌC TRUYỆN TRỰC TUYẾN METRUYEN

Sinh viên thực hiện: Nguyễn Thành Nam

Mã sinh viên: B23DCCN587

Nhóm: 11

Giảng viên hướng dẫn: Kim Ngọc Bách

Hà Nội - 2026

Mục Lục

1. Công việc đã thực hiện trong tuần 3

2.Thiết kế kiến trúc tổng thể 3

3. Thiết kế cơ sở dữ liệu 4

3.1 Đặc tả chi tiết các Collections. 4

3.2. Mối quan hệ giữa các Collection 7

4. Các Usecase trọng tâm 8

UC-01: Đăng nhập hệ thống 8

UC-02: Tìm kiếm và xem danh sách truyện 8

UC-03: Xem chi tiết truyện 9

UC-04: Đọc chương truyện 9

UC-05: Nạp M-Point (Xu ảo) 9

UC-06: Admin – Thêm truyện mới 10

5. Kết quả đạt được 10

6. Kế hoạch tuần tới 10

Tài liệu tham khảo 11

1. Công việc đã thực hiện trong tuần

- Trong tuần thứ 2, em đã tập trung vào việc phân tích và thiết kế hệ thống, cụ thể:

- **Thiết kế kiến trúc hệ thống (System Architecture):**
  - Xác định mô hình Client-Server sử dụng RESTful API.
  - Thiết kế luồng dữ liệu (Data Flow) từ ReactJS (Frontend) xuống NodeJS (Backend) và truy xuất MongoDB (Database).
  - Tích hợp dịch vụ lưu trữ ảnh Cloudinary vào kiến trúc để tối ưu hóa băng thông.
- **Thiết kế Cơ sở dữ liệu (Database Design):**
  - Lựa chọn MongoDB (NoSQL) làm cơ sở dữ liệu chính.
  - Xây dựng chi tiết Schema cho 4 Collections cốt lõi:
    - **Users:** Quản lý thông tin, phân quyền Admin/User và trạng thái VIP.
    - **Categories:** Quản lý danh mục thể loại truyện.
    - **Comics:** Lưu trữ metadata của truyện (Tên, Tác giả, Ảnh bìa, Lượt xem).
    - **Chapters:** Lưu trữ mảng đường dẫn hình ảnh (Image URLs) của từng chương truyện.
    - **Payments:** Lưu trữ thông tin lịch sử giao dịch nạp tiền của member
  - Thiết lập các mối quan hệ (Relationships) giữa các bảng và mô tả các Usecase chính

2.Thiết kế kiến trúc tổng thể

Hệ thống MeTruyen được thiết kế dựa trên mô hình **Client-Server** (Khách - Chủ) kết hợp với kiến trúc **RESTful API**[1] . Để đáp ứng được yêu cầu mở rộng, kiến trúc này được cụ thể hóa thành 4 phân hệ chính:

- **Lớp Client (Frontend / Presentation Layer):** Gửi các yêu cầu HTTP (GET, POST, PUT, DELETE) đến Server để truy xuất hoặc cập nhật dữ liệu. Cụ thể, phân hệ này là một Single Page Application (SPA) được xây dựng bằng **ReactJS** và **TailwindCSS**, chịu trách nhiệm quản lý giao diện người dùng (UI/UX) và hiển thị dữ liệu (Comic Reader, Admin Dashboard).
- **Lớp Server (Backend / Application Layer):** Tiếp nhận yêu cầu, xử lý logic nghiệp vụ, tương tác với Database và trả về kết quả dưới dạng JSON. Phân hệ này được phát triển trên nền tảng **NodeJS** và framework **ExpressJS**, chịu trách nhiệm xác thực người dùng (sử dụng JWT - JSON Web Token), phân quyền (Middleware) và cung cấp các Endpoints[2].
- **Lớp Database (Data Layer):** Sử dụng hệ quản trị CSDL **MongoDB** (NoSQL). Dữ liệu được lưu trữ linh hoạt dưới dạng các tài liệu JSON/BSON, tối ưu hóa cực tốt cho việc lưu trữ cấu trúc mảng lồng nhau của truyện tranh. Tầng Server sẽ giao tiếp với MongoDB thông qua thư viện trung gian ODM **Mongoose** [3].
- **Dịch vụ đám mây (External Services):** Để tối ưu hóa hiệu năng, hệ thống tích hợp dịch vụ **Cloudinary** làm mạng phân phối nội dung (CDN). Hình ảnh truyện tranh tải

trọng lớn sẽ được tải trực tiếp lên Cloud nhằm giảm tải băng thông cho Server chính[4].

- Luồng xử lý yêu cầu (Request Flow) tiêu chuẩn của hệ thống:

1. **Gửi yêu cầu (Client):** Người dùng tương tác trên giao diện (React SPA). Trình duyệt sẽ đóng gói và gửi một HTTP Request (kèm theo JWT Token nếu có) đến các RESTful API của Backend.
2. **Tiền xử lý (Middleware):** Tại Backend, yêu cầu đi qua chuỗi Middleware để kiểm tra bảo mật (CORS), phân tích dữ liệu (Body Parser) và giải mã JWT để xác thực, phân quyền người dùng (Role check).
3. **Xử lý logic (Controller & DB):** Nếu hợp lệ, Controller tiếp nhận yêu cầu, thực thi nghiệp vụ và gọi tới Mongoose Models để thực hiện truy vấn (CRUD) trực tiếp xuống cơ sở dữ liệu MongoDB.
4. **Phản hồi (Response):** Kết quả được Backend đóng gói thành định dạng JSON kèm mã trạng thái HTTP (200, 400, 500...) và trả về. Frontend nhận JSON và dùng Virtual DOM để cập nhật nhanh vùng giao diện thay đổi mà không cần tải lại trang.

3. Thiết kế cơ sở dữ liệu

3.1 Đặc tả chi tiết các Collections.

- Hệ thống bao gồm 5 Collections:

a. Collection Users

- Mô tả: Lưu trữ thông tin người dùng hệ thống

Field	Type	Required	Description
_id	ObjectId	Yes	ID duy nhất (unique)
username	String	Yes	Tên đăng nhập (unique, 3-30 ký tự)
email	String	Yes	Email (unique, validate format)
password	String	Yes	Mật khẩu đã mã hóa (bcrypt, min 6 ký tự)
displayName	String	No	Tên hiển thị (default: username)
avatar	String	No	URL ảnh đại diện
bio	String	No	Giới thiệu bản thân (max 200 ký tự)
role	String	Yes	Vai trò: guest, member, admin
isVIP	Boolean	Yes	Trạng thái VIP (default: false)
vipExpireDate	Date	No	Ngày hết hạn VIP
mPoints	Number	Yes	Xu ảo (M-Point) trong hệ thống
library.bookmarks	Array	No	Danh sách truyện yêu thích
library.history	Array	No	Lịch sử đọc truyện

b. Collection Categories

- Mô tả: Phân loại truyện (Hành động, Tiên hiệp, Võng du, Vô hạn lưu,...).

Field	Type	Required	Description
_id	ObjectId	Yes	ID duy nhất
name	String	Yes	Tên danh mục (unique, max 50)
slug	String	Yes	URL slug (unique, auto-gen)
order	Number	No	Trật tự hiển thị
comicCount	Number	Yes	Số lượng truyện

c. Collection Comics

- Mô tả:Chứa thông tin chung của một bộ truyện.

Field	Type	Required	Description
_id	ObjectId	Yes	ID duy nhất
title	String	Yes	Tiêu đề truyện (max 200 ký tự)
slug	String	Yes	URL-friendly slug (unique, auto-gen)
coverImage.url	String	Yes	URL ảnh bìa (Cloudinary)
coverImage.publicId	String	No	Cloudinary public ID
description	String	Yes	Mô tả/tóm tắt nội dung (max 5000)
author	String	Yes	Tác giả
categories	Array[ObjectId]	No	Mảng ID thể loại (ref: Category)
status	String	Yes	Trạng thái: ongoing, completed, hiatus, cancelled
statistics.totalChapters	Number	Yes	Tổng số chương
statistics.totalViews	Number	Yes	Tổng lượt xem
statistics.totalBookmarks	Number	Yes	Tổng số lượt đánh dấu
createdBy	ObjectId	No	ID Admin tạo (ref: User)
createdAt	Date	Yes	Thời gian tạo
updatedAt	Date	Yes	Thời gian cập nhật

d. Collection Chapters

- Mô tả: Đây là bảng nặng nhất, chứa thông tin các chương truyện

Field	Type	Required	Description
_id	ObjectId	Yes	ID duy nhất
comicId	ObjectId	Yes	ID truyện (ref: Comic)
chapterNumber	Number	Yes	Số thứ tự chương
title	String	No	Tiêu đề chương (max 200 ký tự)
slug	String	Yes	URL slug (auto-gen)
pages	Array[Object]	Yes	Mảng các trang ảnh
pages[].url	String	Yes	URL ảnh trang (Cloudinary)
pages[].publicId	String	No	Cloudinary public ID
pages[].order	Number	Yes	Thứ tự trang
pages[].width	Number	No	Chiều rộng ảnh (px)
pages[].height	Number	No	Chiều cao ảnh (px)
totalPages	Number	Yes	Tổng số trang (auto)
uploadedBy	ObjectId	No	ID Admin upload (ref: User)
previousChapter	ObjectId	No	ID chương trước (ref: Chapter)
nextChapter	ObjectId	No	ID chương sau (ref: Chapter)
createdAt	Date	Yes	Thời gian tạo
updatedAt	Date	Yes	Thời gian cập nhật

e. Collection Payments

- Mô tả: lưu thông tin lịch sử giao dịch nạp tiền của member

Field	Type	Required	Description
_id	ObjectId	Yes	ID duy nhất
userId	ObjectId	Yes	ID người dùng
transactionId	String	Yes	Mã giao dịch (unique)
amount	Number	Yes	Số tiền
status	String	Yes	Trạng thái: pending, success, failed, refunded
paymentMethod	String	Yes	Phương thức thanh toán
createdAt	Date	Yes	Thời gian tạo

3.2. Mối quan hệ giữa các Collection

- Dù MongoDB là hệ quản trị cơ sở dữ liệu phi cấu trúc (NoSQL) lưu trữ dưới định dạng Document, hệ thống vẫn duy trì tính toàn vẹn và liên kết dữ liệu chặt chẽ. Điều này được thực hiện thông qua hai chiến lược mô hình hóa cốt lõi, giúp xử lý linh hoạt các bài toán quan hệ 1-1, 1-N và N-N:

- **Tham chiếu (Referenced Documents):** Sử dụng ObjectId để liên kết các tài liệu độc lập với nhau (tương tự Khóa ngoại trong SQL), tối ưu cho các dữ liệu dùng chung và thường xuyên cập nhật.
- **Nhúng dữ liệu (Embedded Documents):** Lưu trữ trực tiếp tài liệu con vào bên trong tài liệu cha, giúp tối ưu hóa hiệu năng đọc (Read Performance) khi có thể lấy toàn bộ dữ liệu chỉ với một lần truy vấn duy nhất.

STT	A	Quan hệ	B	Cơ chế lưu trữ	Ghi chú
1	User	1 : N	Payment	Payment.userId (Reference)	Một user có nhiều giao dịch nạp tiền
2	User	1 : N	Chapter	Chapter.uploadedBy (Reference)	Một admin upload nhiều chapter
3	User	1 : N	Comic	Comic.createdBy (Reference)	Một admin tạo nhiều truyện
4	User	N : N	Comic	User.library.bookmarks[] (Embedded ObjectId)	Bookmark: user đánh dấu nhiều truyện
5	User	N : N	Comic + Chapter	User.library.history[] (Embedded Object)	Lịch sử đọc: lưu {comicId, chapterId, page}
6	Comic	1 : N	Chapter	Chapter.comicId (Reference)	Một truyện có nhiều chương
7	Comic	N : N	Category	Comic.categories[] (Array of References)	Một truyện thuộc nhiều thể loại
8	Chapter	1 : 1	Chapter	Chapter.previousChapter / nextChapter (Self-Reference)	Liên kết đôi giữa các chương liên tiếp
9	Chapter	1 : N	Page	Chapter.pages[] (Embedded Array)	Các trang ảnh nhúng trực tiếp vào chương

4. Các Usecase trọng tâm

Hệ thống được thiết kế để xử lý nhiều luồng nghiệp vụ phức tạp. Dưới đây là đặc tả chi tiết cho 06 kịch bản Usecases sử dụng quan trọng nhất, bao gồm cả các luồng xử lý ngoại lệ nhằm đảm bảo tính toàn vẹn của hệ thống :

UC-01: Đăng nhập hệ thống

- **Mô tả:** Người dùng xác thực danh tính để nhận quyền truy cập vào hệ thống.
- **Tác nhân:** Guest
- **Tiền điều kiện:** Tài khoản hợp lệ và không bị khóa (isActive = true).
- **Hậu điều kiện:** Frontend nhận JWT token, cập nhật trạng thái đăng nhập và quyền hạn (VIP, số dư M-Point).
- **Luồng sự kiện chính:**
    1. Trình duyệt gửi HTTP POST /api/auth/login kèm email/username và mật khẩu.
    2. Backend tra cứu người dùng trong Database và so sánh hàm băm mật khẩu bằng bcrypt.compare.
    3. Hệ thống kiểm tra trường vipExpireDate, tự động tước quyền VIP nếu đã quá hạn.
    4. Cấp phát JWT (Access Token & Refresh Token) và trả về Client.
- **Luồng thay thế (Alternative Flow):**
    - Nhập sai mật khẩu: Hệ thống trả về lỗi HTTP 401, thông báo chung "Sai email hoặc mật khẩu" để bảo mật thông tin.
- **Luồng ngoại lệ (Exception Flow):**
    - Tài khoản bị khóa: Trả về lỗi HTTP 403 - Tài khoản đã bị vô hiệu hóa.

UC-02: Tìm kiếm và xem danh sách truyện

- **Mô tả:** Người dùng duyệt danh sách truyện, lọc theo thể loại, trạng thái hoặc tìm kiếm từ khóa.
- **Tác nhân:** Tất cả người dùng.
- **Hậu điều kiện:** Hiển thị danh sách truyện, phân trang tối đa 20 kết quả/trang.
- **Luồng sự kiện chính:**
    1. Hệ thống gọi GET /api/comics kèm các tham số truy vấn (query params) như search, category, page.
    2. MongoDB thực thi truy vấn tìm kiếm toàn văn (Text Index) và gọi .populate() để nối bảng lấy tên thể loại.
    3. Frontend nhận mảng dữ liệu và vẽ (render) giao diện lưới danh sách truyện.
- **Luồng thay thế:**
    - Không có truyện thỏa mãn điều kiện lọc: Frontend hiển thị mảng rỗng kèm thông báo "Không tìm thấy truyện phù hợp".
- **Luồng ngoại lệ:**
    - Lỗi mất kết nối: Giao diện hiển thị trạng thái tải ảo (Skeleton Loading) và báo lỗi timeout nếu máy chủ không phản hồi.

UC-03: Xem chi tiết truyện

- **Mô tả:** Truy xuất toàn bộ thông tin mô tả, ảnh bìa và danh sách các chương của một bộ truyện.
- **Tác nhân:** Tất cả người dùng.
- **Hậu điều kiện:** Biến đếm totalViews của truyện tăng thêm 1.
- **Luồng sự kiện chính:**
  1. Hệ thống gửi 2 yêu cầu song song: Lấy thông tin truyện (/api/comics/slug/:slug) và Lấy danh sách chương (/api/comics/slug/:slug/chapters).
  2. Backend gọi hàm findByIdAndUpdate tăng biến đếm lượt xem.
  3. Frontend kiểm tra mảng bookmarks của người dùng (nếu đã đăng nhập) để làm nổi bật (highlight) nút Theo dõi.
- **Luồng thay thế:**
  - Truyện bị ẩn (isPublished = false): Backend chặn truy cập, Frontend điều hướng về trang "Không tìm thấy truyện".
- **Luồng ngoại lệ:**
  - Nhập sai Slug URL: MongoDB trả về kết quả null, Backend phản hồi mã lỗi HTTP 404 Not Found.

UC-04: Đọc chương truyện

- **Mô tả:** Hiển thị nội dung các trang ảnh và xử lý tương tác chuyển chương.
- **Tác nhân:** Tất cả người dùng.
- **Luồng sự kiện chính:**
  1. Trình duyệt gọi API GET /api/chapters/:id.
  2. Backend trả về mảng URL chứa hình ảnh truyện (pages[]) cùng tham chiếu ID của chương trước và chương sau.
  3. Frontend áp dụng cơ chế Lazy Loading để tải tuần tự các trang ảnh nhằm tiết kiệm băng thông.
  4. Nếu người dùng là Thành viên, hệ thống gửi nền (background request) POST /api/users/library/history/:chapterId để lưu vết vị trí đang đọc dở vào CSDL.
- **Luồng thay thế:**
  - Người dùng đang ở chương đầu tiên: Nút "Chương trước" bị vô hiệu hóa (Disabled).
  - Người dùng đang ở chương mới nhất: Nút "Chương sau" bị vô hiệu hóa kèm thông báo "Bạn đã đọc hết chương mới nhất".

UC-05: Nạp M-Point (Xu ảo)

- **Mô tả:** Mô phỏng quy trình nạp tiền thật để nhận M-Point quy đổi trong hệ thống.
- **Tác nhân:** Member
- **Luồng sự kiện chính:**
  1. Người dùng chọn mệnh giá, phương thức thanh toán và nhập Mã giao dịch (Transaction ID).

2. Hệ thống đẩy POST /api/users/recharge-mpoints.
3. Backend kiểm tra tính duy nhất của mã giao dịch, nếu hợp lệ sẽ tạo Document trong bảng payments với trạng thái success.
4. Cộng dồn M-Point vào ví người dùng và phản hồi HTTP 200.

- **Luồng thay thế:**
  - Nhập mệnh giá không hợp lệ: Frontend hiển thị cảnh báo đỏ và khóa nút xác nhận.
- **Luồng ngoại lệ:**
  - Tái sử dụng Mã giao dịch cũ: Ràng buộc Unique Index trên MongoDB sẽ chặn lại, Backend trả về lỗi HTTP 400 - Mã giao dịch đã được sử dụng.

UC-06: Admin – Thêm truyện mới

- **Mô tả:** Quản trị viên tải lên thông tin metadata và ảnh bìa để khởi tạo bộ truyện.
- **Tác nhân:** Admin.
- **Luồng sự kiện chính:**
  1. Admin điền thông tin và nhấn "Tạo truyện".
  2. Payload được gửi đi dưới định dạng multipart/form-data.
  3. Middleware JWT xác thực và Middleware phân quyền Role chặn lại nếu không phải Admin.
  4. Middleware Multer tiếp nhận file ảnh, đẩy trực tiếp lên Cloudinary và trả về đường dẫn URL an toàn.
  5. Backend sử dụng thư viện slugify để tạo tự động đường dẫn thân thiện (URL-friendly) từ Tên truyện và lưu vào MongoDB.
- **Luồng ngoại lệ:**
  - Tên truyện bị trùng lặp dẫn đến lỗi Slug Conflict: Hệ thống tự động xử lý bằng cách thêm hậu tố số (-2, -3) vào chuỗi Slug.
  - Upload ảnh lên Cloudinary thất bại: Backend lập tức rollback, hủy tiến trình tạo truyện và trả về HTTP 500.

5. Kết quả đạt được

- Xây dựng thành công bản vẽ kiến trúc 3 tầng (3-Tier) và luồng xử lý API khép kín.
- Hoàn thiện sơ đồ quan hệ cơ sở dữ liệu với 5 bảng (Collections)
- Đặc tả chi tiết kịch bản sử dụng (Use Cases) cốt lõi của hệ thống.

6. Kế hoạch tuần tới

- Cài đặt môi trường lập trình Backend (NodeJS, ExpressJS, MongoDB).
- Khởi tạo cấu trúc thư mục dự án (Project Structure).
- Code các API cơ bản: Kết nối Database và tạo API Đăng ký/Đăng nhập (Authentication).

Tài liệu tham khảo

[1] "What is RESTful API? - RESTful API Explained - AWS," Amazon Web Services, Inc. Accessed: Mar. 07, 2026. [Online]. Available: https://aws.amazon.com/what-is/restful-api/
[2] "JSON Web Token Introduction - jwt.io." Accessed: Mar. 07, 2026. [Online]. Available: https://www.jwt.io/introduction#what-is-json-web-token
[3] "MongoDB Courses and Trainings | MongoDB University." Accessed: Mar. 07, 2026. [Online]. Available: https://learn.mongodb.com/learn/course/modeling-data-relationships/conclusion/learn?page=2
[4] "Cloudinary Image & Video API Platform - Docs Home Page | Documentation." Accessed: Feb. 27, 2026. [Online]. Available: https://cloudinary.com/documentation